

Final Project: LU Factorization

Nicholas Storti

August 12, 2026

1 Introduction

A common problem in electrical engineering is solving systems of linear equations, which can be represented in the form $\mathbf{Ax} = \mathbf{b}$. In this expression, $\mathbf{A}$ is the matrix of coefficients, $\mathbf{b}$ is a known vector, and $\mathbf{x}$ is the solution vector to be solved for. Systems of linear equations frequently show up in electric and magnetic circuit problems, often involving systems of nodal and loop equations. The solutions to these equations provide values for voltages at nodes and currents through each path. One technique for solving the linear system $\mathbf{Ax} = \mathbf{b}$ is to find the inverse of matrix $\mathbf{A}$ and use the inverse to find $\mathbf{x}$ by:

$$\mathbf{A}^{-1}\mathbf{Ax} = \mathbf{A}^{-1}\mathbf{b}$$

$$\mathbf{Ix} = \mathbf{A}^{-1}\mathbf{b}$$

$$\mathbf{x} = \mathbf{A}^{-1}\mathbf{b}$$

Computing $\mathbf{A}^{-1}$ explicitly, however, is computationally intensive and unnecessary for solving the linear system. A better method is to compute the vector $\mathbf{A}^{-1}\mathbf{b}$ directly, and some techniques for accomplishing this include LU and QR factorization of matrix $\mathbf{A}$.

The topic of this report is a parallel implementation of an LU factorization algorithm. Specifically, the right-looking Doolittle algorithm is implemented in Cuda C to perform parallel LU factorization of matrix $\mathbf{A}$ without pivoting. The report is organized as follows: 2) background section detailing how LU factorization can be used to solve the linear system $\mathbf{Ax} = \mathbf{b}$ and explanation of the right-looking LU factorization algorithm, 3) implementation section detailing three different parallel implementations of the LU factorization algorithm, 4) output section with the output times for different input matrix sizes for each implementation, 5) analysis section where the performance of each implementation is discussed, and 6) conclusion.

2 Background

2.1 Linear System Solution Using LU Factorization

LU factorization is the process of factoring a matrix $\mathbf{A}$ into a lower triangular matrix, $\mathbf{L}$, and an upper triangular matrix, $\mathbf{U}$. For 3x3 $\mathbf{A}$ this looks like:

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ l_{21} & 1 & 0 \\ l_{31} & l_{32} & 1 \end{bmatrix} \begin{bmatrix} u_{11} & u_{12} & u_{13} \\ 0 & u_{22} & u_{23} \\ 0 & 0 & u_{33} \end{bmatrix} = \mathbf{LU}$$

Substituting $\mathbf{A} = \mathbf{LU}$ into $\mathbf{Ax} = \mathbf{b}$ yields [1]:

$$\mathbf{Ax} = \mathbf{b}$$

$$\mathbf{LUx} = \mathbf{b}$$

$$\mathbf{Ux} = \mathbf{L}^{-1}\mathbf{b}$$

$$\mathbf{x} = \mathbf{U}^{-1}(\mathbf{L}^{-1}\mathbf{b})$$

If we let $\mathbf{y} = \mathbf{U}\mathbf{x}$, $\mathbf{A}\mathbf{x} = \mathbf{b}$ becomes [1]:

$$\mathbf{A}\mathbf{x} = \mathbf{b}$$

$$\mathbf{L}\mathbf{U}\mathbf{x} = \mathbf{b}$$

$$\mathbf{L}\mathbf{y} = \mathbf{b}$$

$$\mathbf{U}\mathbf{x} = \mathbf{y}$$

We can then solve for $\mathbf{x}$ by first solving the linear system $\mathbf{L}\mathbf{y} = \mathbf{b}$ for $\mathbf{y}$, then solving the linear system $\mathbf{U}\mathbf{x} = \mathbf{y}$ for $\mathbf{x}$. $\mathbf{L}\mathbf{y} = \mathbf{b}$ can be solved relatively easily using forward substitution since $\mathbf{L}$ is lower triangular. Similarly, $\mathbf{U}\mathbf{x} = \mathbf{y}$ can be solved easily using backward substitution since $\mathbf{U}$ is upper triangular.

This is how the LU factorization of $\mathbf{A}$ can be used to solve the linear system $\mathbf{A}\mathbf{x} = \mathbf{b}$. While the explanation for how the LU factorization can be used is presented, the primary focus of this report is computation of the LU factorization itself on a parallel processor (GPU). The remainder of the report will deal with parallel implementation of an LU factorization algorithm.

2.2 Right-Looking Doolittle LU Factorization Algorithm

The right-looking algorithm is an alternative to Gaussian elimination for calculating $\mathbf{L}$ and $\mathbf{U}$. First, $\mathbf{A} = \mathbf{L}\mathbf{U}$ can be expressed as [1]:

$$\begin{bmatrix} a_{11} & \mathbf{a}_{12} \\ \mathbf{a}_{21} & \mathbf{A}_{22} \end{bmatrix} = \begin{bmatrix} 1 & \mathbf{0} \\ \mathbf{l}_{21} & \mathbf{L}_{22} \end{bmatrix} \begin{bmatrix} u_{11} & \mathbf{u}_{12} \\ \mathbf{0} & \mathbf{U}_{22} \end{bmatrix} = \begin{bmatrix} u_{11} & \mathbf{u}_{12} \\ u_{11}\mathbf{l}_{21} & (\mathbf{l}_{21}\mathbf{u}_{12} + \mathbf{L}_{22}\mathbf{U}_{22}) \end{bmatrix}$$

If $\mathbf{A}$ is $n \times n$, then u_{11} is a scalar diagonal element of $\mathbf{U}$, $\mathbf{u}_{12}$ is a $1 \times n$ row vector of $\mathbf{U}$, $\mathbf{l}_{21}$ is a $n \times 1$ column vector of $\mathbf{L}$, and $\mathbf{L}_{22}$, $\mathbf{U}_{22}$ are $(n-1) \times (n-1)$ trailing submatrices of $\mathbf{L}$ and $\mathbf{U}$. The elements of $\mathbf{L}$ and $\mathbf{U}$ can then be solved for by [1]:

$$u_{11} = a_{11}$$

$$\mathbf{u}_{12} = \mathbf{a}_{12}$$

$$\mathbf{l}_{21} = \frac{1}{u_{11}}\mathbf{a}_{21}$$

$$\mathbf{L}_{22}\mathbf{U}_{22} = \mathbf{A}_{22} - \mathbf{l}_{21}\mathbf{u}_{12}$$

$\mathbf{L}_{22}\mathbf{U}_{22}$ is the Schur complement $\mathbf{S}_{22}$ of $\mathbf{A}_{22}$. The previous calculations can then recursively be used on $\mathbf{S}_{22}$ to find the entirety of $\mathbf{L}$ and $\mathbf{U}$ for $\mathbf{A}$. The steps of the algorithm are:

1. Update current row of $\mathbf{U}$ with current row of $\mathbf{A}$ by $\mathbf{u}_{12} = \mathbf{a}_{12}$.
2. Calculate current column of $\mathbf{L}$ using $\mathbf{l}_{21} = \frac{1}{u_{11}}\mathbf{a}_{21}$.
3. Calculate Schur complement of trailing submatrix using $\mathbf{A}_{22} = \mathbf{A}_{22} - \mathbf{l}_{21}\mathbf{u}_{12}$. Elements of trailing submatrix $\mathbf{A}_{22}$ are overwritten in $\mathbf{A}$ with the Schur complement so that step 1 will be valid for the next iteration.
4. Recursively go back to step 1 on trailing submatrix $\mathbf{A}_{22}$ until all of $\mathbf{L}$ and $\mathbf{U}$ are filled out.

3 Implementation

3.1 Naive Approach

Since each iteration of the algorithm depends on the previous iteration, this aspect must be performed serially and cannot be parallelized. On each iteration, however, there are aspects that can be parallelized. Specifically, the $\mathbf{L}$ column update in step 2 and the Schur complement trailing submatrix update in step 3 on each iteration can be parallelized. Two separate kernels are implemented for each of these calculations.

The first kernel, "luColUpdateKernel_V1" shown below, performs the $\mathbf{L}$ column update in step 2. In this naive implementation, each element of the column is assigned its own thread. Each thread calculates one element in the column of $\mathbf{L}$ for the k^{th} iteration. Over each iteration, the size of the column decreases by 1. So, as the algorithm progresses, fewer threads are active on each iteration. The parameter "size" indicating the dimension of the matrix, the parameter "k" indicating the iteration number, and the thread index are used to determine if a thread is active or not. A major drawback of this implementation is that the number of active threads decreases on each iteration. This leads to increased control divergence as the algorithm progresses and decreased work efficiency.

```
__global__ void luColUpdateKernel_V1(float *out, unsigned int size, unsigned int k) {
    int i = k + 1 + blockIdx.x*blockDim.x + threadIdx.x;

    if(i < size) {
        out[i*size + k] /= out[k*size + k];
    }
}
```

The second kernel, "luTrailingUpdateKernel_V1" shown below, performs the trailing submatrix update in step 3 of the algorithm. In this naive implementation, each element of the output submatrix is assigned its own thread. Each thread calculates one element in the trailing submatrix for the k^{th} iteration. Over each iteration, the dimension of the square submatrix decreases by 1. So, as the algorithm progresses, fewer threads are active on each iteration. The parameter "size", parameter "k", and the thread index are used to determine if a thread is active or not. Just as with the previous kernel, a major drawback of this implementation is that the number of active threads decreases on each iteration. This also leads to increased control divergence as the algorithm progresses and decreased work efficiency.

```
__global__ void luTrailingUpdateKernel_V1(float *out, unsigned int size, unsigned int k) {
    int i = k + 1 + blockIdx.y * blockDim.y + threadIdx.y;
    int j = k + 1 + blockIdx.x * blockDim.x + threadIdx.x;

    if(i < size && j < size) {
        out[i*size + j] -= out[i*size + k] * out[k*size + j];
    }
}
```

Both of these kernels are called within a wrapper function. The wrapper function determines the block and grid dimensions based on the input matrix dimension. The wrapper function uses a for-loop to iterate through the right-looking algorithm. On each iteration, the size of the column of $\mathbf{L}$ being calculated and the dimension of the square trailing submatrix being calculated both decrease by 1. The for-loop iterates until $\mathbf{L}$ and $\mathbf{U}$ have both been calculated. $\mathbf{L}$ and $\mathbf{U}$ are both stored in the output array "out". Since $\mathbf{L}$ is lower triangular and $\mathbf{U}$ is upper triangular, they can be stored in the same array. The diagonal of 1's in $\mathbf{L}$ is then implicit.

3.2 Block Cyclic Coarsening

One way to compensate for decrease in active threads as the algorithm progresses is through thread coarsening with block-cyclic mapping [2][3]. This technique requires a small modification to the right looking

algorithm listed earlier. Step 1 is split into two separate steps where the first element or diagonal element of the current row of $\mathbf{U}$ is calculated separately, then the rest of the row of $\mathbf{U}$ is calculated. The second modification is that instead of individual elements being process by each thread, tiled blocks of the input matrix are processed by each thread to achieve coarsening [2][3].

The modified algorithm consists of four separate kernels. The first kernel performs a complete LU factorization on a diagonal block of the input matrix. The diagonal block and the thread block for this kernel have the same dimensions, so that there will be 1 thread per element of the matrix block. Because of this, the diagonal matrix block will fit entirely in 1 thread block, and the grid for this kernel will always have 1 block. This kernel computes the entire LU factorization on the diagonal block of the input matrix using the right-looking algorithm. On each iteration of the for-loop in the wrapper function, this kernel will progress down the diagonal of the input matrix. On each iteration, this kernel computes the LU factorization of the next diagonal block of the input matrix. For this implementation, the dimension of the diagonal block and the dimension of the thread block are both 16×16 giving a total of 256 threads in the singular thread block for this kernel. Code for "luDiagonalBlockKernel_V2" is shown below.

```
__global__ void luDiagBlockKernel_V2(float *out, unsigned int size, unsigned int k) {
    int tx = threadIdx.x;
    int ty = threadIdx.y;
    int row = k*TILE;
    int col = k*TILE;
    int bs = min(TILE, size - row);

    // LU factorization of A[k][k]
    for (unsigned int p = 0; p < bs; ++p) {
        // Multipliers
        if(tx == p && ty > p && ty < bs) {
            out[(row+ty)*size + (col+p)] = out[(row+ty)*size + (col+p)] / out[(row+p)*size + (col+p)];
        }
        __syncthreads();

        // in place update
        if(ty > p && ty < bs && tx > p && tx < bs) {
            out[(row+ty)*size + (col+tx)] -= out[(row+ty)*size + (col+p)] * out[(row+p)*size + (col+tx)];
        }
        __syncthreads();
    }
}
```

The second kernel completes the rest of the rows of $\mathbf{U}$ for the same rows calculated in the diagonal block kernel on the current iteration. Since the corresponding rows of $\mathbf{L}$ have already been calculated in the diagonal block kernel, and the columns of $\mathbf{A}$ are known, each remaining column j of $\mathbf{U}$ for the required rows can be calculated using forward substitution. Since the diagonal block kernel processes a diagonal block with dimension 16×16 , 16 rows of $\mathbf{U}$ must be completed on each iteration, and the number of columns decreases by 16 on each iteration. In contrast to the diagonal block kernel, multiple blocks are used for this kernel. Each column for the rows of $\mathbf{U}$ to be calculated is assigned to a different block in a cyclic manner. For example, column 1 to block 1, column 2 to block 2, column 3 to block 3, then back around for column 4 to block 1. This distributes the load among the blocks, and it ensures that each block has the same amount of work over each iteration as the number of columns decreases. Each thread in a thread block is assigned its own column to process. Each thread processes the elements in its column sequentially inside a for-loop. This is necessary since forward substitution is sequential. Thus, the columns are processed in parallel, and the rows are processed sequentially. Code for "luRowUpdateKernel_V2" is shown below.

```
__global__ void luRowUpdateKernel_V2(float *out, unsigned int size, unsigned int k, unsigned int num_b,
    int tx = threadIdx.x;
```

```

int ty = threadIdx.y;
int pc = blockIdx.x;
int row = k*TILE;
int bsk = min(TILE, size - row);

// First column > k assigned to this thread block
int j0 = (k + 1) + ((pc - (k + 1)%PGRID_C)%PGRID_C + PGRID_C)%PGRID_C;

// iterate over columns assigned to this thread block
for (unsigned int j = j0; j < num_blocks; j += PGRID_C) {
    int col = j*TILE;
    int bsj = min(TILE, size - col);

    // Forward substitution: solve L*X = A
    // Finding U for the rest of the columns in the rows of A[k][k]
    if(ty == 0 && tx < bsj) {
        for (int p = 0; p < bsk; ++p) {
            float sum = 0.0f;

            for(int t = 0; t < p; ++t) {
                //sum += Lkk_s[p][t] * Akj_s[t][tx];
                sum += out[(row+p)*size + (row+t)] * out[(row+t)*size + (col+tx)];
            }
            //Akj_s[p][tx] -= sum;
            out[(row+p)*size + (col+tx)] -= sum;
        }
    }
    __syncthreads();
}
}

```

Similarly to the row update kernel, the third kernel completes the rest of the columns of $\mathbf{L}$ for the same columns calculated in the diagonal block kernel on the current iteration. Since the corresponding columns of $\mathbf{U}$ have already been calculated in the diagonal block kernel, and the rows of $\mathbf{A}$ are known, each remaining row i of $\mathbf{L}$ for the required columns can also be calculated using substitution. Since the diagonal block kernel processes a diagonal block with dimension 16×16 , 16 columns of $\mathbf{L}$ must be completed on each iteration, and the number of rows decreases by 16 on each iteration. Similarly to the row update kernel, the column update kernel uses multiple blocks. Each row for the columns of $\mathbf{L}$ to be calculated is assigned to a different block in a cyclic matter. Again, this distributes the load evenly among blocks and ensures that each block is performing the same amount of work even when the number of rows decreases over each iteration. Each thread in a thread block is assigned its own row to process. Each thread processes the elements in its row sequentially inside a for loop. This sequential part is necessary since substitution is sequential. Thus, the rows are processed in parallel, and the columns are processed sequentially. Code for "luColUpdateKernel_V2" is shown below.

```

__global__ void luColUpdateKernel_V2(float *out, unsigned int size, unsigned int k, unsigned int num_b)
{
    int tx = threadIdx.x;
    int ty = threadIdx.y;
    int pr = blockIdx.x;
    int row = k*TILE;
    int bsk = min(TILE, size - row);

    // First row > k assigned to this thread block
    int i0 = (k + 1) + ((pr - (k + 1)%PGRID_R)%PGRID_R + PGRID_R)%PGRID_R;

```

```

// iterate over rows assigned to this thread block
for (int i = i0; i < num_blocks; i += PGRID_R) {
    int rowi = i*TILE;
    int bsi = min(TILE, size - rowi);

    // Backward substitution: solve X*U = A
    // Finding L for the rest of the rows in the columns of A[k][k]
    if(tx == 0 && ty < bsi) {
        for(int c = 0; c < bsk; ++c) {
            float sum = 0.0f;

            for(int r = 0; r < c; ++r) {
                sum += out[(rowi+ty)*size + (row+r)] * out[(row+r)*size + (row+c)];
            }
            out[(rowi+ty)*size + (row+c)] = (out[(rowi+ty)*size + (row+c)] - sum)/out[(row+c)*size + (row+c)];
        }
    }
    __syncthreads();
}
}

```

The fourth and final kernel completes the trailing submatrix update for the remaining portion of the input matrix on the current iteration. This kernel computes the Schur complement for the trailing submatrix. Computing the Schur complement is essentially performing a matrix-matrix multiplication with the previously calculated rows of $\mathbf{U}$ and columns of $\mathbf{L}$, then subtracting this from the remaining trailing submatrix of $\mathbf{A}$. Multiple blocks are used for this kernel. The trailing submatrix is divided into smaller blocks, and each matrix block is assigned to a thread block in a cyclic fashion. A 2D grid of thread blocks is used to process the 2D grid of matrix blocks. Assigning matrix blocks to thread blocks in a cyclic manner distributes the load evenly among thread blocks and ensures that each block is performing the same amount of work. This is essential as the trailing submatrix gets smaller on each iteration. Each thread in a thread block is assigned its own matrix block or tile of the trailing submatrix. Each thread calculates the Schur complement for the elements in its tile inside a for loop. Thus, each tiled block of the trailing submatrix is processed in parallel, and the elements of each matrix block are processed sequentially. Assigning entire blocks of elements to each thread achieves coarsening. Code for "luTrailingUpdateKernel_V2" is shown below.

```

__global__ void luTrailingUpdateKernel_V2(float *out, unsigned int size, unsigned int k, unsigned int m) {
    int tx = threadIdx.x;
    int ty = threadIdx.y;
    int pc = blockIdx.x;
    int pr = blockIdx.y;

    int rowk = k*TILE;
    int bsk = min(TILE, size - rowk);

    // first row > k assigned to this thread block
    int i0 = (k + 1) + ((pr - (k + 1)%PGRID_R)%PGRID_R + PGRID_R)%PGRID_R;

    // iterate over rows assigned to this thread block
    for(int i = i0; i < num_blocks; i += PGRID_R) {
        int rowi = i*TILE;
        int bsi = min(TILE, size - rowi);

        // first column > k assigned to this thread block
        int j0 = (k + 1) + ((pc - (k + 1)%PGRID_C)%PGRID_C + PGRID_C)%PGRID_C;
    }
}

```

```

// iterate over columns assigned to this thread block
for (int j = j0; j < num_blocks; j += PGRID_C) {
    int colj = j*TILE;
    int bsj = min(TILE, size - colj);

    // Fill out rest of A[i][j]
    if (ty < bsi && tx < bsj) {
        float sum = 0.0f;

        for(int t = 0; t < bsk; ++t) {
            sum += out[(rowi+ty)*size + (rowk+t)] * out[(rowk+t)*size + (colj+tx)];
        }
        out[(rowi + ty)*size + (colj + tx)] -= sum;
    }
    __syncthreads();
}
__syncthreads();
}
}

```

The four kernels described previously are called within a wrapper function. The wrapper function sets up the block and grid dimensions based on the input matrix dimensions. The wrapper function uses a for-loop to iterate over the right-looking algorithm. On each iteration, the four kernels are called sequentially. Each iteration processes the remaining trailing submatrix from the previous iteration. The for-loop iterates until **L** and **U** have both been calculated. Just as in the naive implementation, **L** and **U** are both stored in the same output array "out".

3.3 Block Cyclic with Shared Memory Tiling

This optimization adds shared memory tiling to the previously described block-cyclic mapping approach. Each of the 4 previously described kernels are modified to utilize shared memory. In each kernel, each thread loads some of the elements of **A**, **L**, and **U** needed for its calculations and stores it in a shared memory array. The loading into shared memory is performed before the main calculations. Each thread collaboratively loads a small portion of the required elements into shared memory so that the entire thread block can access the shared memory array. Each thread in a thread block can then read the elements it needs from shared memory instead of global memory. This reduces global memory accesses and thus reduces bottlenecks associated with multiple threads attempting to access global memory simultaneously.

4 Output

Output times for various steps of execution for each implementation are shown in the following sub sections. Test input matrices of various sizes were used. The input matrices were square, so the listed size is for both dimensions, i.e. $size \times size$. Test input matrices of size 16, 64, 200, 500, 1000, 2000, and 5000 were used. For each size tested, the same input matrix was used between implementations. Further, since pivoting was not implemented, heavy weighting of the diagonals was ensured when generating the test matrices. This ensures numerical stability during floating point calculations, specifically the division calculation in step 2 of the right-looking algorithm.

4.1 Naive Implementation

```

Setting up the problem...0.000011 s
Input size = 16
Allocating device variables...0.130080 s
Copying data from host to device...0.000045 s

```

Launching kernel V1...0.102150 s
Copying data from device to host...0.000022 s
Verifying results...TEST PASSED

Setting up the problem...0.000109 s
Input size = 64
Allocating device variables...0.118778 s
Copying data from host to device...0.000062 s
Launching kernel V1...0.102285 s
Copying data from device to host...0.000028 s
Verifying results...TEST PASSED

Setting up the problem...0.000856 s
Input size = 200
Allocating device variables...0.117829 s
Copying data from host to device...0.000096 s
Launching kernel V1...0.104144 s
Copying data from device to host...0.000176 s
Verifying results...TEST PASSED

Setting up the problem...0.005269 s
Input size = 500
Allocating device variables...0.118814 s
Copying data from host to device...0.000339 s
Launching kernel V1...0.107153 s
Copying data from device to host...0.000940 s
Verifying results...TEST PASSED

Setting up the problem...0.020434 s
Input size = 1000
Allocating device variables...0.118442 s
Copying data from host to device...0.001038 s
Launching kernel V1...0.115034 s
Copying data from device to host...0.002468 s
Verifying results...TEST PASSED

Setting up the problem...0.079680 s
Input size = 2000
Allocating device variables...0.119471 s
Copying data from host to device...0.003546 s
Launching kernel V1...0.153016 s
Copying data from device to host...0.006389 s
Verifying results...TEST PASSED

Setting up the problem...0.495488 s
Input size = 5000
Allocating device variables...0.119847 s
Copying data from host to device...0.021230 s

Launching kernel V1...0.650528 s
Copying data from device to host...0.035788 s
Verifying results...TEST PASSED

4.2 Block Cyclic

Setting up the problem...0.000013 s
Input size = 16
Allocating device variables...0.119421 s
Copying data from host to device...0.000045 s
Launching kernel V2...0.102019 s
Copying data from device to host...0.000023 s
Verifying results...TEST PASSED

Setting up the problem...0.000105 s
Input size = 64
Allocating device variables...0.119145 s
Copying data from host to device...0.000063 s
Launching kernel V2...0.101942 s
Copying data from device to host...0.000028 s
Verifying results...TEST PASSED

Setting up the problem...0.000857 s
Input size = 200
Allocating device variables...0.118103 s
Copying data from host to device...0.000097 s
Launching kernel V2...0.102325 s
Copying data from device to host...0.000184 s
Verifying results...TEST PASSED

Setting up the problem...0.005256 s
Input size = 500
Allocating device variables...0.118829 s
Copying data from host to device...0.000336 s
Launching kernel V2...0.103971 s
Copying data from device to host...0.000945 s
Verifying results...TEST PASSED

Setting up the problem...0.020439 s
Input size = 1000
Allocating device variables...0.118563 s
Copying data from host to device...0.001039 s
Launching kernel V2...0.109097 s
Copying data from device to host...0.002468 s
Verifying results...TEST PASSED

Setting up the problem...0.079582 s
Input size = 2000
Allocating device variables...0.119692 s

Copying data from host to device...0.003542 s
Launching kernel V2...0.138167 s
Copying data from device to host...0.006389 s
Verifying results...TEST PASSED

Setting up the problem...0.495276 s
Input size = 5000
Allocating device variables...0.119584 s
Copying data from host to device...0.021197 s
Launching kernel V2...0.643908 s
Copying data from device to host...0.036078 s
Verifying results...TEST PASSED

4.3 Block Cyclic with Shared Memory Tiling

Setting up the problem...0.000013 s
Input size = 16
Allocating device variables...0.118991 s
Copying data from host to device...0.000045 s
Launching kernel V3...0.102091 s
Copying data from device to host...0.000023 s
Verifying results...TEST PASSED

Setting up the problem...0.000104 s
Input size = 64
Allocating device variables...0.118916 s
Copying data from host to device...0.000064 s
Launching kernel V3...0.101908 s
Copying data from device to host...0.000027 s
Verifying results...TEST PASSED

Setting up the problem...0.000858 s
Input size = 200
Allocating device variables...0.118214 s
Copying data from host to device...0.000098 s
Launching kernel V3...0.102157 s
Copying data from device to host...0.000184 s
Verifying results...TEST PASSED

Setting up the problem...0.005280 s
Input size = 500
Allocating device variables...0.119162 s
Copying data from host to device...0.000339 s
Launching kernel V3...0.103160 s
Copying data from device to host...0.000945 s
Verifying results...TEST PASSED

Setting up the problem...0.020447 s
Input size = 1000

```

Allocating device variables...0.118276 s
Copying data from host to device...0.001038 s
Launching kernel V3...0.106526 s
Copying data from device to host...0.002469 s
Verifying results...TEST PASSED

```

```

Setting up the problem...0.079643 s
Input size = 2000
Allocating device variables...0.119735 s
Copying data from host to device...0.003546 s
Launching kernel V3...0.126763 s
Copying data from device to host...0.006377 s
Verifying results...TEST PASSED

```

```

Setting up the problem...0.495335 s
Input size = 5000
Allocating device variables...0.119686 s
Copying data from host to device...0.021225 s
Launching kernel V3...0.394529 s
Copying data from device to host...0.035879 s
Verifying results...TEST PASSED

```

4.4 Summary of Results

A summary of the execution times for each kernel is shown in Table 1 below. These results are shown in tabular form as opposed to graphical form because most of the values are very close. The execution times between implementations are very close, and the differences would not be easily visible on a plot. A table allows for the exact difference in time values between implementations to be seen.

Table 1: Kernel Execution Times (seconds)

N	Naive (s)	BC (s)	BC Shared Mem. (s)
16	0.102150	0.102019	0.102091
64	0.102285	0.101942	0.101908
200	0.104144	0.102325	0.102157
500	0.107153	0.103971	0.103160
1000	0.115034	0.109097	0.106526
2000	0.153016	0.138167	0.126763
5000	0.650528	0.643908	0.394529

A plot of memory transfer time from host to device as a function of input matrix dimension is shown below in Fig. 1. Both x and y axes are in logarithmic scale. It can be seen from the plot that for input dimension 200×200 and below, the memory transfer time increases slowly as the matrix size increases. However, above 200×200 , the memory transfer time increases at a higher rate. Since it is linear on the log plot, the transfer time increases exponentially with input matrix dimension. A possible explanation for this sharp increase above a certain matrix dimension is bandwidth limitations of the memory hardware. The hardware can only transfer so much memory at one time. Above a certain size threshold, the memory will have to be transferred in multiple rounds instead of all at once. The sharp increase in transfer time above 200×200 indicates that this is the threshold for a single memory transfer. Above this size, the memory needs to be transferred in multiple steps.

A plot of memory transfer time from device to host as a function of input matrix dimension is also shown below in Fig. 2. For this plot as well, both x and y axes are in logarithmic scale. The memory transfer time

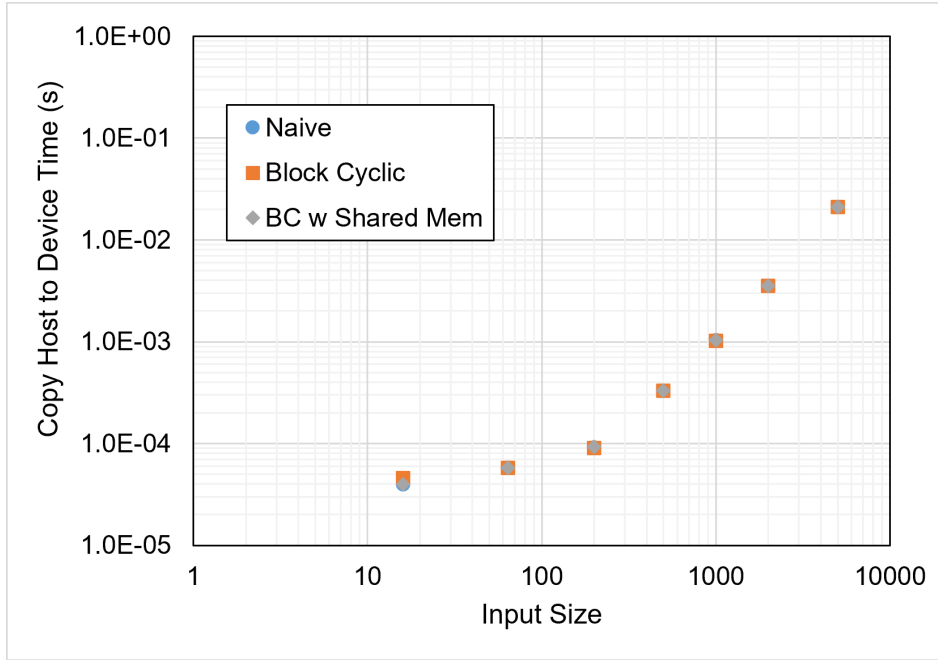


Figure 1: Graph of memory transfer time from host to device vs. input size.

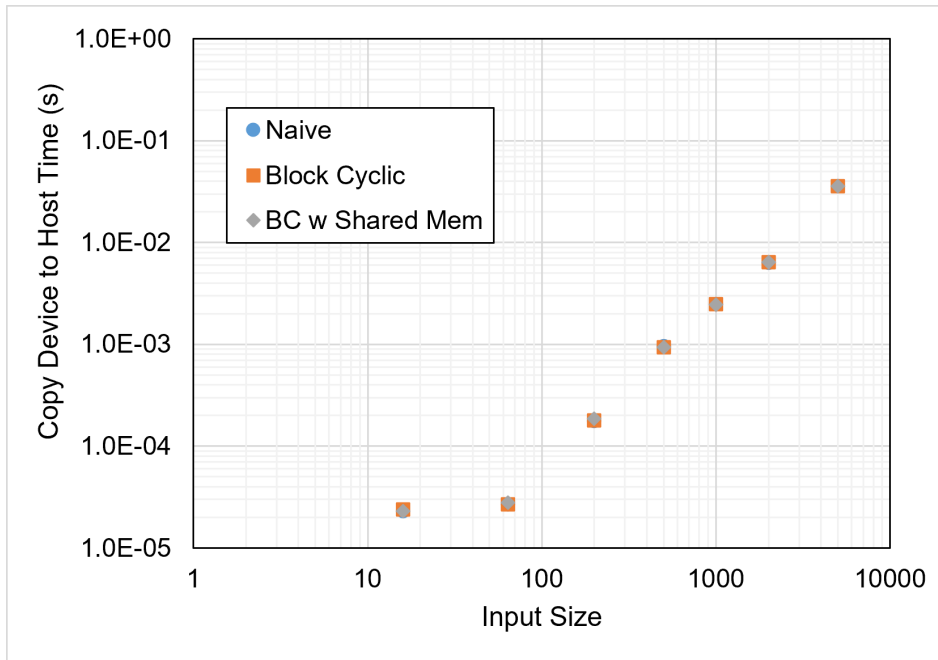


Figure 2: Graph of memory transfer time from device to host vs. input size.

from device to host also shows a small increase with matrix size below matrix dimension 200, and shows an exponential increase in memory transfer time for matrix dimension above 200.

Regarding hardware utilization, all thread blocks for each kernel are made to have 256 threads. For the instructional GPUs, the streaming multiprocessors (SM) can have a maximum of 1536 threads. With thread blocks of size 256 threads, 6 thread blocks can fit evenly on 1 SM. If 6 or a multiple of 6 thread blocks are used, this yields 100% utilization of the SMs. The number of blocks used for each kernel is determined by the input matrix size.

5 Analysis

Starting with a comparison between the naive and block-cyclic implementations, it can be seen from Table 1 above that for all test matrix sizes, the block-cyclic approach showed faster execution times than the naive approach. Part of this speedup can be attributed to the coarsening achieved by the block-cyclic approach. Because each thread performs some sequential work, this reduces the total number of iterations through the right looking algorithm called by the outer loop in the wrapper function. Thread coarsening also helps with memory latency. Since each thread is performing more work, the elements it needs for calculations are stored in registers. It can easily access the data it needs from registers for multiple calculations instead of having to go to global memory. In the naive approach, each thread only computes one output element, and it has to go to global memory each time for the elements needed for the calculation. Further, the block-cyclic mapping ensures that the load is evenly distributed among the thread blocks, even as the number amount of total work reduces through each iteration. In the naive approach, all of the work gets concentrated in the same block or blocks as the algorithm progresses, which results in poor utilization in later iterations. However, the block-cyclic approach ensures even load across the thread blocks even during the later stages of the right-looking algorithm, resulting in more even utilization.

Regarding the block-cyclic approach with shared memory, it can be seen from Table 1 that this approach achieves the fastest execution times for all input test matrix sizes except for size 16. For size 16, the shared memory implementation out performs the naive implementation, but it is slightly slower than the block-cyclic implementation without shared memory. For size 16, in both block-cyclic implementations, only the diagonal block kernel will be called, since it processes a diagonal block of size 16×16 . This kernel can process the entire input if it is only size 16. In the block-cyclic approach with shared memory, it is possible that for an input matrix of small size 16, the overhead needed for writing to shared memory, then copying the result from shared memory to global memory outweighs the advantages of using shared memory. The block-cyclic implementation without shared memory does not have to deal with this overhead. So, for the small input size of 16, the block-cyclic implementation without shared memory executes faster. For all other sizes, however, the block-cyclic implementation with shared memory executes faster than both the naive approach and the block-cyclic approach without shared memory. The reasons for the faster execution speed than the naive approach has been discussed previously for the block-cyclic approach without shared memory. The primary reason for the increase in speed between block-cyclic with and without shared memory is the use of shared memory. Using shared memory for calculations reduces global memory accesses. Global memory accesses are slow, and they are subject to collisions when multiple threads are trying to access simultaneously. Shared memory is much quicker for the threads to access, so using it for large input matrix sizes results in substantial execution speed increases.

6 Conclusion

The goal of this project was to implement LU factorization on a parallel processor, specifically using the right-looking Doolittle algorithm. Three different approaches were compared. A naive implementation, a block-cyclic mapping implementation, and a block-cyclic with shared memory implementation were each presented. The execution speed results of each implementation were also compared and discussed. It was found that the most optimized approach, block-cyclic mapping with shared memory, performed the fastest for all input matrix sizes above 16×16 . All source code files, report files, and presentation files can be found under the "Final" directory in the following GitHub repository: <https://github.com/1995mazdamiata/ELC5319.git>

7 References

1. <https://courses.grainger.illinois.edu/cs357/sp2020/notes/ref-9-linsys.html>
2. <https://www.netlib.org/utk/papers/outofcore/paper.html>
3. https://solomonik.cs.illinois.edu/teaching/cs554_fall2017/slides/slides06.pdf